

Divide and Conquer (Closest Pair of points) ①

The problem that we are going to discuss now comes from computational geometry. We will use the same algorithm design strategy here.

So the problem is this

Input: Set of points in the plane

output: closest pair of points

The points are represented by their x and y ~~coordinates~~ co-ordinates. There are n such points with their x and y coordinates and we are interested to find ~~two~~ a pair of points which are closest to each other among all possible pairs of points.

So given any two points the distance is $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$, the usual notion of the distance that we have in the plane. So for each pair of points there is a distance and there are nC_2 pair of points, and so nC_2 distances we want to pick the minimum distance

The obvious way to do it is to compute the distance between every pair of points and find the minimum

(2)

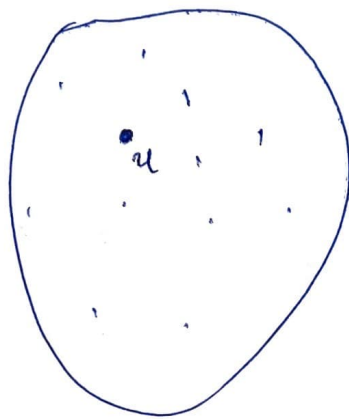
Since there are nC_2 distances, the algorithm takes nC_2 time.

Now the obvious question is can we do better. do we really need to compute all nC_2 distances? Well that is a interesting question here and the answer is No. and we will do it much faster than n^2 . So somehow ~~you~~ we will not be computing all nC_2 distances, that is the moral of the story

Let us now see how we go about designing the algorithm. Well if we think a bit, you start wondering ^{to} why should we need to compute every pair of distances. Can we look at the geometry of the points and come up with an idea that ~~at~~ avoid computation of nC_2 distances. Can we also think about inductive approach, which says suppose I remove one point from the set and add it ~~so~~ back so that we can get rid of some points! may be yes may be no but this is something that we need to think about when ~~is~~ we get a new problem to solve.

Let us look at this approach this will not be the approach that we will finally follow but this is something that I wanted you to think about for every problem. ③

So, the classic set of induction approach to this or any such problem is that you remove a point, recurse on the rest, put the point back and see what you get.



So, let us ~~remove~~ the point u have been removed from the region shown, and some how I find the minimum distance among the remaining point.

Now when I put back the point u in the region what will be the minimum distance? Obvious thing is to compute ^{the} distance of all other point from this point u to find the new minimum distance.

But this is not the efficient way to do that as there are $n-1$ other point in the region and we need to compute $n-1$ distances.

(4)

The recurrence would ~~look so~~ like lead to a $O(n^2)$ solution as the recurrence would look like

$$T(n) = (n-1) + T(n-1)$$

and this recurrence if you solve you get

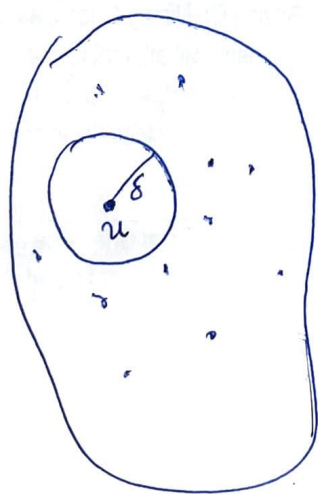
$$T(n) = O(n^2).$$

well, do I need to compute $(n-1)$ distances when I put the point u back in the region?

Can we not get rid of some of the points?

This is the crucial set of observation here.

Suppose the minimum distance in the region except the point u is δ . The distance δ is calculational recursively. One crucial observation



is that any points that are in contention are the points in a circle of radius δ around the point u . The other points are, well, too far away.

If you can quickly compute the distance of this points

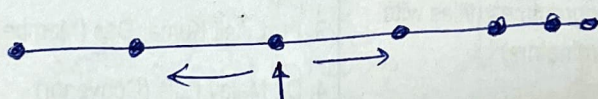
in the circle for u then you are in the business, I mean you have achieved something.

$$T(n) = \underline{(n-1)} + T(n-1)$$

(n-1) components come from the calculation of distance after pulling back the point u in the region. So if we can compute this much faster may be in $\text{constant}/\log n$ time then we are in a good shape

So you should be thinking in this line, how do we cut $(n-1)$ down to something else.

So before we go further let us do this in one dimension. You are given a set of points on a line, and you want to find the closest pair.



How will you do this? Well I think you all should see that we don't need to compute all n^2 distances. Well the important thing to notice here is that, if you are looking at a particular point the only candidates are the adjacent points. So for any point we need to check only two distances. and this gives us $2n$ distances to be checked. But which two distances of a point?

(6)

So in one dimension what we can do is sort them, and once we sort them we look at their increasing order of coordinate from left to right, and for each point we look at the distance from the previous and next point, calculate the distances and choose the minimum. So the algo becomes

1. dimension

— Sort

— Choose the min by seeing from left to right.

Now how much time does this algorithm take?

Sorting ~~sorting~~ takes $n \log n$ time.

So we started with n^2 distances but time taken is $O(n \log n)$. Sorting dominates this procedure if we first sort it then it takes $n \log n$ time. We have reduced it from n^2 to $n \log n$.

There are some morals of the story.

First one you always try simpler problem. You are given this problem in two dimension. It always pays to check first check what the problem means in one dimension. You hear a little bit

(7)

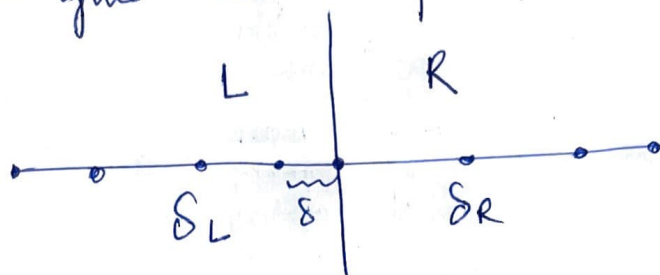
it is the first thing and you atleast now know that maybe it is possible to do it in less than n^2 time. Because you certainly can do it in one dimension in $n \log n$ time.

Question now one ask is, what about two dimension? Is there something like sorting that we can do which ultimately helps us.

The answer is actually yes. and lets see how this is done. It is quite a smart algorithm and we will apply divide and conquer approach reasonably blindly. But as we proceed we will use some trick that ~~seep~~ speed up the algorithm. So the first step is something you can think of is I want to divide the part into two parts, find the minimum on the left part, find the minimum on the right part. Then once we find the minimum on both the part we use these minimum to compute the overall minimum. Before we do this I think it will be a bit illustrative to do this on the line.

In the earlier case on the line we first sorted the points and have seen that it took $n \log n$ time. Now let us see what this gives us. divide and conquer techniques gives us.

we are given this n points. The divide and



conquer strategy will divide the points into two parts one on the left and one on the right. recurse on both. Once get the min distance both part, the only thing that we have not checked is the min distance between points on the left and the points on the right. That is we need to check the distance between the last point on the left and the first point on the right and check if this distance is less than the min that we have already computed. So let say δ_L and δ_R are the respective min on the left and right and let say the min distance that we have just computed is δ . and the overall min

(9)

It be the min of this three. I need to make sure that I find the ~~to~~ last point on the left and the last point on the right recursively. So while applying the divide and conquer we not only find the minimum distances on both halves but also we find the last point of the left half and ~~as~~ the last point on the right half. and once we do this we can put these thing together and get a solution. The partition could be done by finding the median which takes linear time. So if we look at the recurrence it looks like

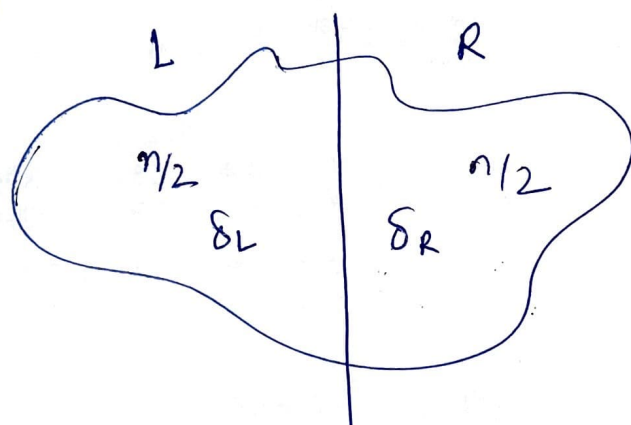
$$T(n) = 2T(n/2) + O(n) \text{ and this we}$$

know is $n \log n$.

This does not really use sorting in any sense except finding the median. ~~that takes linear time.~~

Now let us do this on the plane. We are given points on the plane and we are going to divide it into two parts

Let us divide the point set as based on x -coordinate.
 The leftmost $n/2$ points form one partition and the
 rightmost $n/2$ points form another partition. Recursively
 the minimum distance is found out on the left
 and right half, (i.e. the closest pair of points on
 the left half and right half). Now we need to
 find the closest pair for the entire set of points.
 Our goal is to ~~see~~ find it as fast as possible.



δ_L min distance on
the left
 δ_R min distance on
the right

let

$$\delta = \min(\delta_L, \delta_R)$$

Now one important observation is that if at all there
 are candidates or closest pair it has to be that
 one point has to be on the left and one point
 has to be on the right. Any candidate pair
 has to be of this form, they can not both be on
 the left and both be on the right, one of them ~~has~~
 has to be on the left and the other has to be on
 the right.

What else can be said? The recurrence can be written as

$$T(n) = 2T(n/2) + \text{something}$$

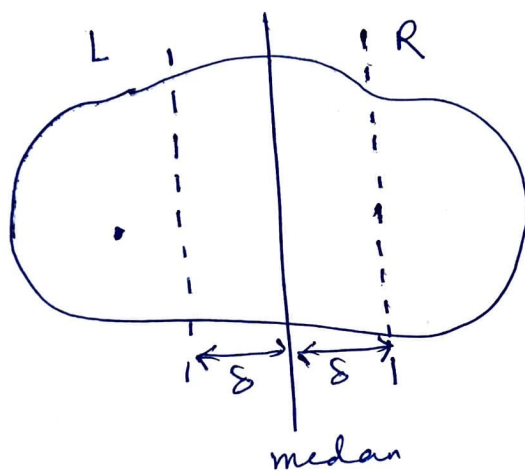
If the other thing is order n then this recurrence is $O(n \log n)$ and we are in good shape. So this for the other thing, the amount of work that we will need to find the closest pair one from the left and the other from the right.

The natural way of doing this is to just pick every on the left, every point on the right and find the distance every pair of points. There are $n/2$ points on the left and $n/2$ points on the right and that gives rise to $\frac{n^2}{4}$ pairs. So if we compute $\frac{n^2}{4}$ distances then I am gone. That's too much to do. In fact we will be back to computing n^2 distances. So this is not what we want to do. So how can we speed things up? This is the question that we need to answer. Now at this point it is left to you ingenuity, left to little bit of luck, but there are no clear tricks, anything that I can teach you, which you can use. Just left to your own intelligence and perseverance.

12

You can try small problems, small example etc etc but, the idea from this part onwards are completely new. They are problem dependent and there are no general techniques that I can tell you which you can use.

The first thing to note, is where do the candidate points lie? let us look at the figure

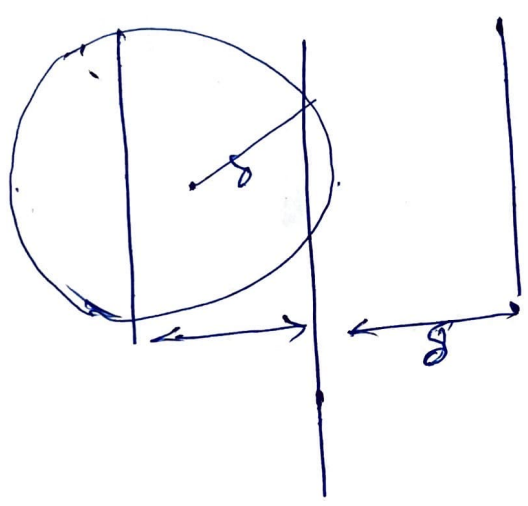


The candidate points have to be within a distance δ from the median. If a point lies outside this region the distance between them would be greater than δ . So I can forget about all those point lies outside the δ bands on both side

The only points that are of interest to us are those which lie on the small band around the middle line.

But just this is not enough. The same problem can arise ~~as~~ in the sense that all $n/2$ points can lie in the left band and all $n/2$ points ~~be on~~ on the right lies on the right band and in this case we are back to computing n^2 distances.

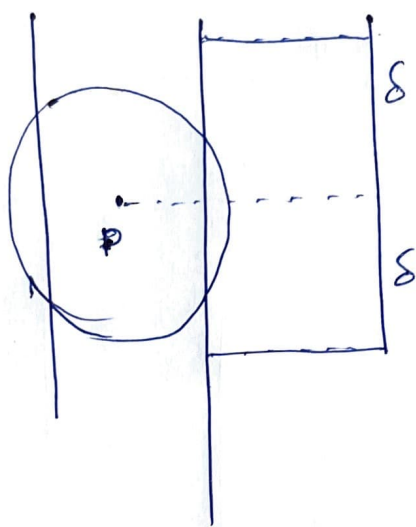
Here is the other smart thing, let us magnify the figure, the small band around the circle line.



If we take any point on the left, and draw a circle of radius g around it, there will be no point inside the circle. The reason is that the minimum distance is g and if a point lies inside the circle the min distance will be less than g . So these points are well spread, there can't be many points clustered together

They are all spread nicely and the same is true on the right side. We would like to use this fact to speed up the algorithm.

What else can be said? Well let us look at the point on the left.



What about the corresponding point on the right? Well, the candidate points will be over the square of side δ . So any candidate points for p lies on the square on the right. There should not be many points on the square also.

How many points can be there in each square. Intuitively there can not be many points

The algorithm

(15)

Step 1 Initialization

Sort the points on X coordinate and
on Y coordinate { we have two arrays
one for X , and one for Y . }

- This initialization is done before the algorithm starts. and these two arrays will be passed to each recursive steps

Step 2 Divide step

Let the value of the median (when sorted on X -coordinate) be X_{med}

- Split X into X_L and X_R

points in X_L have their X -coordinate $\leq X_{med}$, other points are in X_R

- split Y into Y_L & Y_R

Y_L contains the same points as X_L
sorted on Y -coordinate

Y_R - similarly

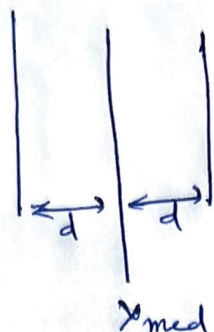
The input is divided into two parts $X_L Y_L$ and $X_R Y_R$

Step 3 Recurse step

d_L : distance of the closest pair on input $X_L Y_L$

d_R : same on (X_R, Y_R)

$$d = \min(d_L, d_R)$$



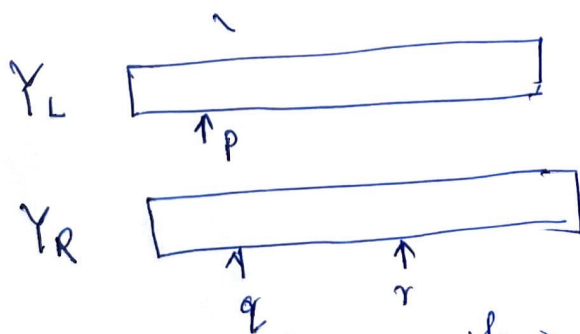
Step 4: Putting them together

X'_L : points from X_L whose x coordinate are
 $\geq x_{med} - d$

Y'_L : y coordinates of the same set of points
 as in X'_L sorted on y -coordinate
 This is done by proving

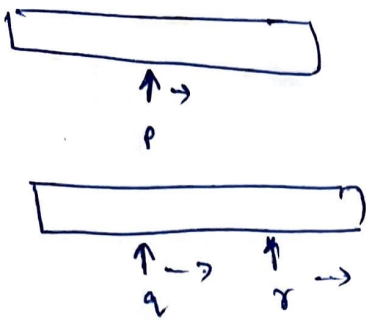
X'_R : points from X_R with x coordinate
 $\leq x_{med} + d$

Y'_R : points as in X'_R sorted on y coordinate



Maintain these pointers as shown

for each point (x, y) in Y'_L we will find (at most 6)
 points whose y coordinates are in the range $(y' - d)$ to $(y' + d)$



Between q and r we find the parts with correct range of y coordinate

$$p = p + 1$$

now q appropriately
" r "

range $y' - d$ to $y' + d$
increased as long as
 $y \text{ coordinate} < y' - d$
decreased as long as y
coordinate $> y' + d$.

Compute distances of $y'_L(p)$ & all points between
 $y'_R(q)$ and $y'_R(r)$

compute the min of these distances say δ

$$\text{return } \delta \text{ min}(\delta, d)$$

end.

Analysis

$$\text{Sorting: } O(n \log n)$$

then two recursive call

$$2T(n/2) + O(n)$$